TY-CODE: ARCHITECTURE & TECHNICAL INTERVIEW GUIDE

Senior Developer and Software Architect Role Interview Preparation.
System Design, Tech Stack Rationale, API Normalization, Security Sandbox, and Production Use Cases.

---

1. The Executive Pitch and Product Vision

The 30-Second Elevator Pitch

"Ty-Code is a local-first, standalone AI coding IDE built on Electron, React 18, and Monaco Editor. Unlike cloud-locked tools such as Cursor or Copilot, Ty-Code uses a zero-trust, Bring-Your-Own-Key model. It provides a unified multi-provider gateway. This gateway connects to proprietary cloud APIs, local offline LLMs via Ollama, and decentralized free tiers via OpenKilo and FCC. The platform includes automated quota balancing, local credential encryption using OS safeStorage, and a secure agent execution sandbox."

The 2-Minute Architectural Pitch (For Directors and Architects)

"I designed Ty-Code to address three major problems in current AI developer tools: vendor lock-in, unpredictable costs, and cloud privacy risks.

The system uses a three-tier desktop architecture:

1. Renderer Process: Built with React 18, Vite, and Zustand. It hosts the Monaco Editor, an XTerm.js terminal emulator, and an interactive M3E UI Canvas sketcher running in a separate window.

2. Preload and ContextBridge Layer: A strictly typed IPC boundary. It uses Zod schemas to validate all messages and prevent prototype pollution.

3. Main Process and Agent Core: A Node.js engine managing native pseudo-terminals via node-pty, background Language Server Protocol daemons, a ReAct agent execution loop, and a unified ProviderGateway.

The gateway normalizes different streaming protocols into one standard stream. An adaptive router inspects task complexity. It sends reasoning tasks to large models and fast edits to local or low-latency models. Source code and credentials remain encrypted locally and never leave the machine in plaintext."

> **KEY TAKEAWAY: Core Value Proposition**
> Ty-Code proves that an AI coding assistant can deliver high productivity without sending private code to a third-party cloud. It provides complete data sovereignty and unified multi-model routing.

2. Technology Stack Selection and Technical Rationale

Q: Why Electron instead of a Web IDE or a VS Code Extension?

Focus Area: Platform Architecture

Building Ty-Code as an Electron application was a deliberate choice for three reasons:

1. Direct OS-Level Privileges: Web browsers cannot spawn native pseudo-terminals (node-pty). They cannot run Language Server Protocol binaries or access local files without constant permission prompts. Electron provides full native capabilities.

2. Data Sovereignty and Offline Use: Enterprise teams with sensitive code cannot send files through remote proxy servers. Electron allows Ty-Code to connect directly to local models (such as Ollama or vLLM) on localhost with zero internet access required.

3. Why Not a VS Code Extension? VS Code extensions run inside a restricted sandbox. They cannot control window lifecycles, create separate windows like our M3E Canvas, customize window chrome, or enforce custom OS-level security policies.

Q: Why Monaco Editor instead of CodeMirror or Ace Editor?

Focus Area: Editor Architecture

Monaco Editor was selected for three clear reasons:

1. Developer Familiarity: Monaco is the editor engine inside VS Code. Developers already know its keybindings, multi-cursor editing, minimap, and bracket matching. There is no learning curve.

2. Built-in DiffEditor: When the agent proposes code edits over 5 lines, Ty-Code requires visual review. Monaco includes a production-grade DiffEditor that shows side-by-side and inline visual diffs with individual chunk acceptance.

3. Language Server Integration: Monaco aligns directly with Language Server Protocol data structures. This simplifies showing real-time diagnostics, auto-completions, and type definitions.

Q: Why Zustand for state management instead of Redux Toolkit or React Context?

Focus Area: State Management

Zustand was chosen over Redux and Context due to performance during high-speed AI streaming:

1. No Cascading Re-renders: Language models stream at 50 to 150 tokens per second. React Context re-renders every consuming component on every token update. Zustand uses atomic selector subscriptions. Only the specific status badge or token counter updates.

2. Out-of-React State Dispatch: Zustand stores can be read and updated outside React components via getState() and setState(). In an Electron app where IPC events arrive asynchronously, this avoids React hook closure bugs.

3. Small Footprint: Zustand is under 1KB gzipped, keeping bundle size minimal.

Q: Why Vite and esbuild for the build pipeline?

Focus Area: Build Engineering

Ty-Code uses a dual build setup:

• Renderer Process: Vite 6 and React 18 use native ES modules. This delivers instant Hot Module Replacement during development.

• Main and Preload Processes: Bundled with esbuild targeting Node 20 CommonJS. esbuild compiles native dependencies and TypeScript in milliseconds while leaving native Node addons external.

This configuration guarantees sub-second build times across both processes.

3. Deep-Dive Technical Architecture and Engineering Solutions

Q: How does the Unified Provider Gateway normalize different streaming APIs?

Focus Area: API Gateway and Normalization

Different AI providers use incompatible streaming formats. OpenAI and Groq emit Server-Sent Events. Anthropic uses typed delta events. Ollama and local servers emit NDJSON lines.

We built ProviderGateway using the Adapter Pattern. Outbound requests are converted into standard message schemas. Inbound streams are transformed into unified TypeScript AgentStreamChunk events:

• 'token': Incremental text deltas.
• 'thought': Reasoning tokens for models such as DeepSeek R1 or OpenAI o1.
• 'tool_call': Structured tool execution requests.
• 'status': Execution state transitions (for example, 'Thinking...').

If a provider fails or hits rate limits (HTTP 429), the gateway automatically fails over to the next compatible model in the pool.

```
// Normalized Agent Stream Event Schema
export type AgentStreamChunk =
  | { taskId: string; type: 'token'; content: string }
  | { taskId: string; type: 'thought'; content: string }
```

```
  | { taskId: string; type: 'status'; content: string }
  | { taskId: string; type: 'tool_call'; toolCall: ToolCallDefinition }
  | { taskId: string; type: 'approval_request'; approvalRequest: AgentApproval }
  | { taskId: string; type: 'diff_proposed'; diff: FileDiffPayload }
  | { taskId: string; type: 'done'; usage?: TokenUsageSnapshot }
  | { taskId: string; type: 'error'; error: string };
```

Q: How do you prevent high-frequency streaming tokens from freezing the UI?

Focus Area: Performance Engineering

Fast models like Groq can emit over 300 tokens per second. Firing a React state update for each token saturates the JavaScript event loop and causes dropped frames.

We implemented a double-buffer flush system in the renderer:

1. Micro-Batching: Incoming tokens accumulate in an in-memory string buffer.
2. 24ms Throttle Window: A requestAnimationFrame throttle synchronizes UI updates with the 60Hz display refresh cycle.
3. Deferred Markdown Parsing: Raw text displays immediately during streaming. Full syntax highlighting and markdown parsing run only after the stream finishes.

Q: How do you minimize Time-to-First-Token (TTFT) and optimize end-to-end streaming latency?

Focus Area: Latency Optimization & Network Architecture

To deliver instantaneous visual feedback without compromising throughput, Ty-Code optimizes latency across three critical tiers:

1. Main Process Outbound Coalescing with Immediate Chunk #1 Flush:
Tokens stream from AI providers through the Main process. Instead of blasting IPC messages per token, tokens coalesce into 16–24ms batches. Crucially, Chunk #1 bypasses batching and flushes synchronously across IPC in 0ms. This ensures Time-to-First-Token (TTFT) is minimal while eliminating IPC serialization overhead during sustained 300+ token/s bursts.

2. Per-Host HTTPS Connection Pooling (keepAlive: true):
ProviderGateway maintains persistent HTTPS agents and Undici dispatchers per provider host with keepAlive: true, 15-second keepalive timeouts, and socket reuse. Sequential prompts reuse active TLS sessions and TCP connections, saving 80 to 250ms of handshake latency per request.

3. CSS Layout Containment & Virtualization:
Message rows are virtualized via @tanstack/react-virtual and isolated with React.memo. Each message container applies content-visibility: auto and contain-intrinsic-size: auto 130px. Off-screen conversation turns completely bypass layout and paint calculations until scrolled into view.

4. Deferred AST Syntax Highlighting:

During live streaming, messages render raw text through ScrambleText. Heavy markdown AST construction and highlight.js tokenization run only once upon receiving the [DONE] stream termination token.

Q: Explain the Smart Sticky-Scroll Auto-Follow algorithm and how you prevented layout jumps and per-pixel re-renders.

Focus Area: UI Engineering & DOM Synchronization

Implementing auto-scrolling during high-speed AI streaming is fraught with edge cases: programmatic smooth scrolling queues animations, deferred markdown highlighting shifts container heights, and reading scroll position in React state causes catastrophic per-pixel re-renders.

We designed a multi-layer Smart Sticky-Scroll architecture in ChatMessageList:

1. Ref-Based Pin Detection (Zero Re-render Cost):
We track pinned state using const pinnedToBottom = useRef(true). A { passive: true } scroll listener throttled by requestAnimationFrame computes distanceFromBottom = el.scrollHeight - el.scrollTop - el.clientHeight. The user is considered pinned when distanceFromBottom < 64px. A single piece of React state (detachedFromBottom) flips ONLY when the boolean changes value, guaranteeing zero React re-renders while scrolling.

2. Auto-Follow on Buffer Flush (Behavior: 'auto'):
After each 24ms double-buffered flush commits text to the DOM, if pinnedToBottom.current is true, el.scrollTop = el.scrollHeight executes using behavior 'auto'. Programmatic smooth scrolling is strictly prohibited during streaming because queued browser animations cause visual jitter and artificial lag.

3. ResizeObserver Height Protection:
A ResizeObserver monitors the inner virtual container. When post-stream events (syntax highlighting on [DONE], code block expansion, image loading) increase document height while pinned, the observer re-pins scrollTop immediately. If the user is detached (reading previous messages), the observer exits immediately, preventing jarring layout jumps.

4. Floating 'Jump to latest' Pill & Re-Pin Events:
When detachedFromBottom is true, a floating pill with an ArrowDown icon and backdrop blur appears bottom-center. Clicking the pill or submitting a new message forces pinnedToBottom.current = true and triggers a smooth scroll to the bottom. Scrolling manually back within 64px automatically hides the pill.

Q: How does the Matrix Decode glyph-scramble animation work, and how did you guarantee zero latency?

Focus Area: Animation Performance & Accessibility

To provide a tactile visual aesthetic during token streaming, we built a 'Matrix Decode' glyph-scramble animation (ScrambleText).

1. Provably Zero Latency:
The animation never gates, buffers, delays, or re-requests tokens. It is strictly a decoupled presentation transform operating on text already committed to the render buffer. Tokens are painted into memory immediately.

2. Per-Character Dynamic Lock Formula:
When character index i first appears in the text buffer, it is assigned a lock timestamp:
lockAt = now + 90ms + (i % 5) * 12ms
On each animation frame, characters where lockAt <= now render as their real glyph. Unlocked characters render a pseudo-random glyph from a Katakana and hex charset ('アイウエオカキクケコサシスセソ0123456789ABCDEF#$%&'). Whitespace and newlines are never scrambled to prevent word-wrap shifts.

3. Bounded O(1) Trailing Window:
To prevent CPU degradation on long responses, scrambling is strictly bounded to the trailing 200 characters (SCRAMBLE_WINDOW = 200). All older characters render permanently locked with zero per-frame CPU computation.

4. Lifecycle & Accessibility Guards:
• Single Self-Cancelling rAF Loop: Starts only when unlocked characters exist and auto-terminates when all characters lock or upon unmount.
• Tab Visibility Guard: Automatically pauses the rAF loop when document.hidden is true.
• Reduced Motion: Respects prefers-reduced-motion: reduce by immediately rendering locked text.
• Accessibility: Uses aria-label={text} and aria-live='off' so screen readers receive clean canonical prose without scrambled characters.

Q: How does Ty-Code enforce enterprise typography and code-safe UK English punctuation?

Focus Area: Typography & Natural Language Normalization

Enterprise communication demands consistent typography, rich markdown presentations, and professional punctuation style without corrupting source code.

1. Zustand Typography System (useTypographyStore):
Exposes persistent settings for chatFontFamily ('sans', 'serif', 'mono', 'plex'), chatFontSize (12–18px, default 14), matrixDecodeEnabled, and ukPunctuationEnabled. Subscribed via atomic selectors to eliminate unnecessary component re-renders.

2. Rich Markdown Enhancements:
Rendered with custom components for alternating zebra-striped tables with sticky headers, task lists with styled interactive checkboxes, KaTeX mathematical expressions, and Mermaid architectural diagrams.

3. System Prompt UK English Rules:
Injected across all provider prompts: standard UK spellings (colour, organise, analyse, favour), eliminating em dashes in prose, and favoring short, direct clauses with commas, semicolons, or colons.

4. Code-Safe Punctuation Normaliser (normalisePunctuation):
Post-processing AI output requires extreme caution so that source code is never mutated. The normaliser parses markdown into fenced code blocks (```...```), inline code spans (`...`), HTML tags, and prose segments. Em dashes (—) and spaced en dashes (–) are replaced with commas or hyphens strictly in prose segments. Fenced code, CLI flags (--option), language constructs (x--), and numeric ranges (1999–2005) remain 100% immutable.

Q: Explain Ty-Code's local-first security sandbox and permission tiers.

Focus Area: Application Security

To allow agents to work autonomously without risking system damage, Ty-Code uses three security layers:

1. Terminal Execution Policy:
- Always Ask: The agent pauses and asks for user approval before running any command.
- Restricted Mode: Safe inspection commands (ls, cat, git status) run automatically. Dangerous commands (rm, npm install, curl) require approval.
- Always Proceed: Autonomous execution mode for sandboxed testing.

2. Workspace Containment Sandbox: File operations (read, write, edit) are restricted to the open workspace folder. Directory traversal attempts are blocked in the main process unless the user explicitly enables System Access in Global Settings.

3. Safe Edit Review Gate: Any edit modifying more than 5 lines triggers an interactive Monaco visual diff review before changing files on disk.

Q: How are API keys and credentials protected at rest on the client machine?

Focus Area: Credential Cryptography

Storing API keys in plain text files (.env or JSON) creates serious security vulnerabilities. Ty-Code integrates Electron's native safeStorage API:

- Windows: Encrypted with Windows DPAPI (Data Protection API) tied to the OS user account.
- macOS: Encrypted with Apple Keychain Services.
- Linux: Encrypted using the Secret Service API (GNOME Keyring or KWallet).

Keys are stored on disk as encrypted binary buffers. They are decrypted only in memory within the Main process when making HTTPS requests. The Renderer process receives only masked previews (for example, 'sk-ant-...1234') and never handles plaintext keys.

Q: How is Inter-Process Communication designed to prevent memory leaks?

Focus Area: IPC Architecture

1. Process Isolation: nodeIntegration is set to false, and contextIsolation is set to true.
2. Typed contextBridge: The preload script exposes a locked window.electronAPI interface. Renderer code cannot access raw Node modules.
3. Disposable Listener Pattern: Every stream listener returns an explicit cleanup function. React components run this cleanup during unmounting. This prevents detached DOM nodes and listener memory leaks.
4. Zod Schema Validation: All parameters crossing the IPC bridge are verified with Zod schemas to reject invalid payloads.

Q: How does the Multi-Window Architecture work for the M3E UI Sketcher Canvas?

Focus Area: Window Management

The M3E UI Sketcher needed an open visual canvas that would take too much space inside the main editor.

In src/main/index.ts, the openSketcherWindow() function manages a secondary BrowserWindow. It loads the application bundle with a '#sketcher' hash route.

Window controls (minimize, maximize, close) are custom styled for consistency. They send IPC commands targeting the specific window instance using BrowserWindow.fromWebContents(event.sender).

State synchronization between the editor and canvas runs through an IPC event bus (canvasSyncManager). This enables real-time visual UI generation from chat prompts.

Q: What is the Context Window Auto-Compaction strategy?

Focus Area: Context Management

In long multi-step coding workflows, token accumulation can quickly exceed model context limits or create high costs.

Ty-Code tracks the token ratio against the model limit. When usage reaches 95% capacity, the agent executes an Auto-Compaction pass:

1. The original system prompt and project requirements remain permanently pinned.
2. Intermediate file outputs and test logs are condensed into a compact semantic summary.
3. The last 2 conversation turns are kept intact for ongoing context.

This recovers roughly 65% of the token budget while keeping the agent fully informed.

4. System Design and Scalability Interview Questions

Q: How would you scale Ty-Code across an Enterprise Organization of 1,000+ developers?

Focus Area: Enterprise Architecture

Scaling Ty-Code across an enterprise relies on four clear pillars:

1. Centralized Policy Distribution: Distribute standardized enterprise policy files enforcing Restricted Mode terminal policies, restricting file access strictly to workspace folders, and disabling telemetry.

2. Private LLM Gateway Integration: Point client installations to an internal corporate proxy (such as a vLLM or LiteLLM cluster) inside the private cloud. The proxy manages corporate billing and audits, while Ty-Code connects via standard endpoints.

3. Air-Gapped Binary Distribution: Support internal package repositories and offline auto-update channels for secure workstations.

4. Audit Logging: Save all terminal executions and file changes to a local, tamper-evident log for compliance reviews.

Q: How do you guarantee zero telemetry leakage and client privacy?

Focus Area: Privacy and Compliance

Ty-Code enforces a strict Zero-Telemetry policy:

1. No Silent Tracking: Outbound telemetry is disabled by default. If enabled for local diagnostics, events are stored strictly in an encrypted local file on the user's machine.

2. Telemetry Inspector: Users can review every stored diagnostic event in the Privacy tab. No destination URL is configured in the codebase, so nothing is transmitted externally.

3. Continuous CI Guard Rails: Automated scripts run on every pull request. They verify that no tracking pixels, analytics packages, or unencrypted credentials exist in the codebase.

5. Real-World Use Case Scenarios and Business Impact

Enterprise Scenario	Ty-Code Architectural Capability	Tangible Business Value
Regulated Environments (Healthcare, Defense, Banking)	Local Ollama or vLLM integration, safeStorage encryption, and Zero-Telemetry guarantee.	Enables AI coding assistance in air-gapped environments where proprietary code cannot leave the local network.
Cost-Sensitive Startups and Students	One-Click Free Mode routing through OpenKilo (40+ models) and the FCC proxy layer.	Removes subscription barriers ($20 to $40 per user monthly), providing complete agent capabilities at zero cost.
Multi-Cloud Reliability	Unified ProviderGateway with dynamic fallback across OpenRouter, Groq, Anthropic, and Gemini.	Prevents developer downtime during outages by automatically switching to available alternative models.
Rapid UI Prototyping	Standalone M3E Canvas Window with React Flow and bidirectional chat synchronization.	Accelerates design-to-code workflows by converting visual layouts directly into production React and Tailwind code.
Multi-Language Monorepo Development	Native Language Server Protocol client supporting TypeScript, Python, Rust, and Go.	Delivers compiler-grade diagnostics and code completions inside Monaco without heavy third-party IDE setups.

6. Engineering Gotchas and Hard Problems Solved

Q: What was the most challenging technical hurdle on this project?

Focus Area: Technical Challenge and Ground Truth

"The hardest technical challenge was guaranteeing real-world execution integrity across asynchronous streaming and native terminals.

Many open-source AI boilerplates use mock timers, simulated delays, or fake streams. We took two major steps:

1. Automated CI Guard Suite: Scripts run in CI to scan code syntax trees. They block any hardcoded replies or fake delays in production agent paths.

2. Live Connectivity Tracer: We built a runtime tracer that measures real round-trip latencies across every hop (Renderer -> IPC Bridge -> Gateway -> Agent Loop -> Network Provider). It provides live latency data and error reporting directly to the developer.

In addition, building cross-platform terminal support with node-pty required custom native bindings to bridge Windows ConPTY and Unix pseudo-terminals cleanly."

Q: Why is programmatic smooth scrolling dangerous during high-speed AI streaming?

Focus Area: DOM Animation Queueing & Compositor Performance

"A critical engineering gotcha we resolved was smooth scrolling animation queueing.

When an AI model streams at 150 to 300+ tokens per second, token buffer commits occur every 20 to 24ms. If the scroll container uses CSS 'scroll-behavior: smooth' or if code calls el.scrollTo({ behavior: 'smooth' }), the browser's layout engine attempts to interpolate between dozens of rapid height updates. These smooth scroll animations queue up in the compositor, creating severe frame drops, visual rubber-banding, and detached scroll states where the viewport lags far behind the live token stream.

We solved this by strictly enforcing behavior: 'auto' (instantaneous assignment) during all streaming flushes, setting .chat-feed { scroll-behavior: auto; }, and reserving smooth scrolling solely for discrete, user-initiated actions (such as clicking the 'Jump to latest' pill or submitting a new message)."

7. Interview Quick-Reference Cheat Sheet

- **Architecture Pattern:** Multi-Process Electron Architecture (Main, Preload, Renderer) with strict Process Isolation and contextBridge.
- **Design Patterns Used:** Adapter Pattern (ProviderGateway), ReAct Loop (Agent Engine), Observer and Disposable Pattern (IPC Streaming), Atomic Selector Subscriptions (Zustand), Self-Cancelling Animation Loop (ScrambleText).

- **Key Protocols:** Server-Sent Events (SSE), JSON-RPC 2.0 (Language Server Protocol), ANSI escape sequences (XTerm.js and node-pty), Electron IPC, HTTPS keepAlive connection pooling.
- **Security Standards:** OS safeStorage (DPAPI, Keychain, libsecret), Three-Tier Terminal Sandbox (Always Ask, Restricted, Always Proceed), Workspace Path Traversal Containment.
- **Performance Optimizations:** Main Process token coalescing (16–24ms) with immediate Chunk #1 flush (0ms TTFT); Double-buffered 24ms rAF renderer flush; Smart sticky-scroll auto-follow with <64px pin detection and zero-per-pixel re-renders; ResizeObserver height-jump isolation; Bounded 200-char Matrix Decode glyph scramble; CSS content-visibility: auto layout virtualization; Per-host HTTPS keepAlive socket reuse; Token auto-compaction at 95% threshold; Monaco Diff review gating.
- **Typography & Text Processing:** Modular Zustand typography system (System Sans, Georgia Serif, JetBrains Mono, IBM Plex Mono); Code-safe UK English punctuation AST normaliser protecting fenced code blocks, inline spans, and CLI syntax.